

```
In [ ]: import marimo as mo
```

Introdução

Neste notebook construímos um gerador/resolvedor de Sudoku $n^2 \times n^2$ modelado como um **problema de satisfação de restrições (CSP)**. A ideia central é que a regra do Sudoku é sempre a mesma (um conjunto de células tem de ter valores todos diferentes) e só muda que células pertencem ao conjunto. Por isso tudo é um **grupo de células**: linhas, colunas, blocos e pistas aleatórias são grupos, e o modelo CSP trata-os todos da mesma forma, sem saber de que tipo são.

Correspondência entre os requisitos do enunciado e os nomes usados no código:

Requisito	Nome no enunciado	Nome no código
R1	box	Grupo
R2	cube	Cube
R3	path	Path
R4	pistas aleatórias	pistas
R5	modelo e resolução	SudokuCSP
R6	Sudoku completo	sudoku

Organização: Cada requisito tem uma célula com o código, uma célula de teste e uma célula com a justificação. No fim juntamos tudo (R6), validamos automaticamente a solução com $n = 2$ e $n = 3$ e mostramos a grelha.

R1- Box

```
In [ ]: class Grupo:
    """conjunto de células com a restrição 'todos diferentes'.
    Guarda (linha, coluna)->valor fixo, ou None se a célula for livre."""

    def __init__(self, n, cells=None):
        self.n = n
        self.N = n * n
        self.cells = {}
        if cells is not None:
            for (i, j), val in cells.items():
                self.add(i, j, val)

    def add(self, i, j, val=None):
        if not (0 <= i < self.N and 0 <= j < self.N):
            raise ValueError(f"Célula ({i}, {j}) fora da grelha")
        if val is not None and not (1 <= val <= self.N):
            raise ValueError(f"Valor {val} fora de [1, {self.N}]")
```

```

        self.cells[(i, j)] = val

    def matrix(self):
        m = [[0] * self.N for _ in range(self.N)]
        for (i, j), val in self.cells.items():
            if val is not None:
                m[i][j] = val
        return m

```

Célula de teste de R1, o resultado é o esperado. Como o enunciado pede para que o notebook teste automaticamente com n diferentes, achamos que valia a pena acrescentar o caso n=2 à célula de teste R1.

```

In [ ]: _g = Grupo(3)
        _g.add(0, 0, 5) #adiciona celula (0,0) com 5
        _g.add(1, 1)   #célula livre

        _erros = []
        for _i, _j, _v in [(9, 0, None), (0, 0, 10), (-1, 2, 3)]: #o none pertence
            ao grupo mas n tem pista/valor fixo
            #o 10 n é valido pq nao existe, 0-9; linha -1 é valida
            try:
                _g.add(_i, _j, _v)
                _erros.append(f"({_i},{_j},{_v}) não foi rejeitado")
            except ValueError:
                pass

        _g2 = Grupo(2) #com 2x2=4 as linhas validas sao 0-3 e os valores 1-4
        _g2.add(3, 3, 4) #tem de aceitar
        try:
            _g2.add(4, 0) #rejeta pq n existe linha 4
            _erros.append("(4,0) com n=2 NÃO foi rejeitado")
        except ValueError:
            pass

        mo.callout(
            mo.md(f"cells = `{_g.cells}` \nprimeira linha = `{_g.matrix()[0]}`"
                + (" " if not _erros else f" \n**Falhas:** {_erros}")),
            kind="success" if not _erros else "danger",
        )

```

```
cells = {(0, 0): 5, (1, 1): None}
```

```
primeira linha = [5, 0, 0, 0, 0, 0, 0, 0, 0]
```

R1: Justificação

Correspondência com o enunciado: `box` → `Grupo`. Escolhemos este nome porque a classe representa um *grupo de células* sujeito à restrição "todos diferentes", e não a grelha em si.

Ideia central: Num Sudoku a regra é sempre a mesma: um conjunto de células tem de ter valores todos diferentes. O que muda entre uma linha, uma coluna e um bloco é apenas *que células pertencem ao conjunto*. Por isso `Grupo` só sabe guardar "um conjunto de células, algumas já fixas", e não sabe nada sobre linhas, colunas, blocos ou Sudoku. É esta generalidade que permite tratar todos os grupos da mesma forma e acrescentar variantes (diagonais, regiões irregulares) sem alterar o modelo.

Estrutura de dados: Usamos um dicionário `(linha, coluna) → valor | None`:

- a chave identifica a célula e o valor distingue *célula fixa* (inteiro) de *célula livre* (`None`), que é exatamente a informação que o enunciado pede;
- só guarda as células que pertencem ao grupo. Um grupo tem tipicamente n^2 células, muito menos do que as n^4 da grelha, por isso uma matriz $n^2 \times n^2$ por grupo desperdiçaria memória;
- acesso e inserção em tempo médio constante, e uma célula repetida limita-se a sobrescrever a entrada anterior em vez de ficar duplicada.

Parametrização: A classe guarda `n` e calcula `N = n2` uma única vez. Todos os limites (coordenadas em $[0, N - 1]$, valores em $[1, N]$) dependem de `N`, e nunca de um `9` escrito no código. Isto garante que funciona para qualquer n (testado com $n = 2$ e $n = 3$).

Validação: O método `add` rejeita, com `ValueError`, coordenadas fora da grelha e valores fora de $[1, n^2]$, e só depois guarda a célula. Assim, um grupo nunca fica num estado inválido. O construtor, quando recebe um conjunto inicial de células, chama `add` para cada uma, de modo que a validação existe num único sítio e não há maneira de contornar as regras pelo construtor.

Representação matricial: `matrix()` devolve uma matriz $n^2 \times n^2$ com o valor fixo nas células fixas e `0` em todas as outras (células livres e células fora do grupo). Aqui perde-se a distinção entre "livre" e "não pertence" sendo intencional: a matriz serve para mostrar valores, e a distinção continua disponível no dicionário `cells`, que é o que o modelo CSP usa.

R2: Cube

```
In [ ]: class Cube(Grupe):
    """bloco nxn de índices (bi, bj). Canto superior esquerdo em (bi*n,
    bj*n)"""

    def __init__(self, n, bi, bj):
        super().__init__(n) # cria grupo vazio chamando o construtor grupo
        if not (0 <= bi < n and 0 <= bj < n): #bi e bj sao indices do
            raise ValueError(f"Bloco ({bi}, {bj}) fora de [0, {n})")
        for di in range(n):
            for dj in range(n):
```

```

        self.add(bi * n + di , bj * n + dj)

        #cube(3, 1, 2) por ex, cria um sudoku 9x9 e queremos o bloco
        com indices(1,2)

```

Célula de teste R2

```

In [ ]: _b = Cube(3, 1, 2)
assert len(_b.cells) == 9
assert min(i for i, _ in _b.cells) == 3 and max(i for i, _ in _b.cells) ==
5
assert min(j for _, j in _b.cells) == 6 and max(j for _, j in _b.cells) ==
8

_b2 = Cube(2, 1, 1)
assert sorted(_b2.cells) == [(2, 2), (2, 3), (3, 2), (3, 3)]

try:
    Cube(3, 3, 0)
    _rejeitou = False
except ValueError:
    _rejeitou = True
assert _rejeitou

mo.callout(mo.md("`Cube` OK para $n=3$, $n=2$ e rejeita blocos
inválidos."), kind="success")

```

Cube OK para $n = 3$, $n = 2$ e rejeita blocos inválidos.

R2: Justificação: Cube (bloco $n \times n$)

Ideia principal: Um bloco $n \times n$ é só um grupo de células, por isso `Cube` herda de `Grupo`. A herança é adequada porque um `Cube` é um `Grupo` (tudo o que se pode fazer com um grupo faz-se com um bloco) e porque o modelo CSP vai tratá-lo exatamente como qualquer outro grupo, sem precisar de saber que é um bloco.

Construção: O bloco de índices (b_i, b_j) , com $0 \leq b_i, b_j < n$, tem o canto superior esquerdo em $(b_i \cdot n, b_j \cdot n)$. O construtor chama `super().__init__(n)` para criar o grupo vazio e depois percorre os deslocamentos $d_i, d_j \in [0, n)$, adicionando a célula $(b_i \cdot n + d_i, b_j \cdot n + d_j)$. São n^2 células, que é precisamente o tamanho de um bloco e o número de valores distintos que o bloco tem de conter.

Reutilização e validação: `Cube` não repete nenhuma lógica de `Grupo`: usa `add`, que já valida as coordenadas. Acrescentamos apenas a validação de (b_i, b_j) , porque sem ela um índice de bloco inválido seria rejeitado pelo `add` com uma mensagem sobre uma *célula* fora da grelha.

Generalidade: Nada no código assume $n = 3$: os índices e o tamanho do bloco dependem só de n . Para $n = 2$ obtêm-se 4 blocos 2×2 numa grelha 4×4 , e para $n = 3$ obtêm-se 9 blocos 3×3 numa grelha 9×9 (ambos verificados nos testes).

R3: Path

```
In [ ]: class Path(Grupe):
    """troço reto (horizontal ou vertical) de início a fim, em qualquer
    sentido"""

    def __init__(self, n, início, fim):
        super().__init__(n)
        (i0, j0), (i1, j1) = início, fim
        if i0 != i1 and j0 != j1: #verifica se linhas = ou colunas =,
            horizontal ou vertical, não permite diagonal
            raise ValueError("início e fim têm de estar na mesma linha ou
            coluna")

        def _sinal(x):
            return (x > 0) - (x < 0)

        di, dj = _sinal(i1 - i0), _sinal(j1 - j0) #em q direção andar, pos
        1 neg -1, zero 0
        #se andamos linha ou coluna, funciona em qq sentido

        tamanho = max(abs(i1 - i0), abs(j1 - j0)) + 1 #quantas células
        entre início e fim, incluindo os dois
        for k in range(tamanho): #vai adicionando as células ao grupo
            self.add(i0 + k * di, j0 + k * dj)
```

início = (2,1) fim = (2,5)

di = 0 dj = 1 tamanho = 5 k=0 → (2,1) k=1 → (2,2) k=2 → (2,3) k=3 → (2,4) k=4 → (2,5)

Célula de teste R3

```
In [ ]: #horizontal, sentido inverso
assert list(Path(3, (0, 5), (0, 2)).cells) == [(0, 5), (0, 4), (0, 3), (0,
2)]
# vertical, sentido normal
assert list(Path(3, (2, 1), (5, 1)).cells) == [(2, 1), (3, 1), (4, 1), (5,
1)]
#célula única
assert list(Path(3, (4, 4), (4, 4)).cells) == [(4, 4)]
#linha completa com n=2
assert len(Path(2, (1, 0), (1, 3)).cells) == 4

for _ini, _fim in [((0, 0), (1, 1)), ((0, 0), (0, 9))]: # diagonal, fora
da grelha
    try:
        Path(3, _ini, _fim)
        raise AssertionError(f"{_ini}->{_fim} devia ser rejeitado")
    except ValueError:
```

pass

```
mo.callout(mo.md("`Path` OK nos dois sentidos, horizontal e vertical, e  
rejeita casos inválidos."), kind="success")
```

Path OK nos dois sentidos, horizontal e vertical, e rejeita casos inválidos.

R3: Path (troço reto de células)

Ideia central: Uma linha, uma coluna, ou qualquer troço reto de células, é apenas um grupo de células. Por isso `Path`, tal como `Cube`, herda de `Grupo` e limita-se a preencher as células: o modelo CSP trata-o como qualquer outro grupo, sem precisar de saber que é um troço reto.

Construção: Dados `inicio = (i0, j0)` e `fim = (i1, j1)`, o troço inclui ambos os extremos. Em vez de tratar separadamente os quatro sentidos possíveis (direita, esquerda, baixo, cima), calculamos o passo em cada eixo como o **sinal** da diferença:

$$d_i = \text{sgn}(i_1 - i_0), \quad d_j = \text{sgn}(j_1 - j_0)$$

O sinal vale 1, 0 ou -1 , e o troço tem $\max(|i_1 - i_0|, |j_1 - j_0|) + 1$ células. Se `fim` vem *depois* de `inicio`, os passos são positivos; se vem *antes*, são negativos e o troço é percorrido em sentido inverso, como o enunciado exige. Se os dois pontos estão na mesma linha, $d_i = 0$ e só a coluna varia (e vice-versa), pelo que não é preciso um `if` por direção.

Casos limite:

- Se `inicio = fim`, o tamanho é 1 e o grupo tem uma única célula.
- Se os pontos não estão na mesma linha nem na mesma coluna (uma diagonal), o construtor levanta `ValueError`. Sem esta verificação, o código construiria uma diagonal sem avisar, o que seria um erro silencioso.
- Coordenadas fora da grelha são rejeitadas por `add`, herdado de `Grupo`, por isso a validação de limites não é repetida.

Utilização no Sudoku: A linha i é `Path(n, (i, 0), (i, N-1))` e a coluna j é `Path(n, (0, j), (N-1, j))`, com $N = n^2$. Há n^2 linhas e n^2 colunas, e cada uma tem n^2 células. Como os limites dependem só de N , funciona para qualquer n (testado com $n = 2$ e $n = 3$).

R4: Pistas escolher células aleatórias e atribuir valores aleatórios válidos, criando grupo com essas pistas

```
In [ ]: import random

def pistas(n, k=None, rng=random):
```

```

N = n * n

if k is None: #k= nr de pistas, sao celulas com nr ja atribuidos que o
CSP tem de respeitar
    k = n

if not (0 <= k <= N):
    raise ValueError(f"k deve estar entre 0 e {N}") #entre n ao
quadrado

posicoes = [ #p/ n=3 temos 81 posicoes possiveis
    (i, j)
    for i in range(N)
    for j in range(N)
]

escolhidas = rng.sample(posicoes, k) #escolhe aleatoriamente
valores = rng.sample(range(1, N + 1), k) #sample é sem repeticao, k
valores entre 1 e N2

g = Grupo(n) #guarda num grupo

for (i, j), v in zip(escolhidas, valores): #zipa cada posicao com um
valor
    g.add(i, j, v) #guarda

return g #retorna as pistas

```

Célula Teste R4

```

In [ ]: for _n in (2, 3):
    _g = pistas(_n)
    assert len(_g.cells) == _n # k por omissão = n
    assert all(v is not None for v in _g.cells.values())
    _vals = list(_g.cells.values())
    assert len(set(_vals)) == len(_vals) # valores distintos
    assert all(1 <= v <= _n * _n for v in _vals)
    assert len(pistas(3, k=9).cells) == 9 # k máximo, so podemos
    escolher 9 pistas
    try:
        pistas(3, k=10)
        raise AssertionError("k=10 devia ser rejeitado")
    except ValueError:
        pass
    mo.callout(mo.md("`pistas` OK para $n=2$ e $n=3$."), kind="success")

```

pistas OK para $n = 2$ e $n = 3$.

R4: pistas aleatórias

Resultado: `pistas(n, k)` devolve um `Grupo`, sem classe nova: o enunciado pede apenas que o resultado seja, de novo, um grupo.

Escolha das células: Usamos `random.sample` sobre a lista de todas as posições da grelha, que escolhe k posições *sem repetição*. Se sorteássemos com repetição, a mesma célula poderia sair duas vezes e `add` limitar-se-ia a sobrescrevê-la, ficando o grupo com menos de k pistas.

Escolha dos valores: O modelo impõe "todos diferentes" a cada grupo, e o grupo de pistas é um grupo como os outros. Se dois valores sorteados fossem iguais, o puzzle ficaria **sempre** sem solução, qualquer que fosse a posição das células. Por isso sortecemos também os valores sem repetição, o que exige $k \leq n^2$ (e `pistas` rejeita k fora de $[0, n^2]$). Esta é uma restrição ao enunciado ("valor aleatório em $[1, n^2]$ "): os valores continuam aleatórios, mas distintos.

Valor por omissão: $k = n$, da ordem de n como o enunciado sugere. Com poucas pistas o puzzle tem muitas soluções.

Puzzle sem solução: Mesmo com valores distintos, não é garantido que o puzzle seja solúvel para qualquer k (as posições escolhidas podem forçar conflitos indiretos). Optámos por **reportar o insucesso**: o sudoku devolve `None` como solução e o teste falha com uma mensagem clara, em vez de repetir o sorteio até haver solução. Preferimos assim porque não esconde um problema no modelo (se o puzzle falhar muitas vezes, queremos vê-lo), e tentar novas pistas até dar poderia demorar muito para n grandes, onde cada resolução é cara. Como o sorteio é aleatório, basta voltar a correr a célula para obter novas pistas.

- **R5: Modelo CSP**

```
In [ ]: class SudokuCSP:
    """modelo CSP para uma grelha  $n^2 \times n^2$ ."""

    def __init__(self, n):
        self.n = n
        self.N = n * n

        #grupos adicionados ao modelo, restricoes
        self.groups = []

    def add_group(self, group):
        """adiciona um grupo"""
        if not isinstance(group, Grupo): #path, cube ..., garantir q é
            #classe q herda de grupo tipo path e cube
            raise TypeError("O grupo tem de ser uma instância de Grupo")

        self.groups.append(group)

    def _neighbors(self, cell):
        #devolve as células que têm restrição com a célula dada, ve os
        #vizinhos

        neighbors = set()
```

```

    for group in self.groups:
        if cell in group.cells:
            neighbors.update(group.cells)

    neighbors.discard(cell)

    return neighbors

def _fixed_values(self):
    #Obtem os valores fixos pelas pistas

    fixed = {}

    for group in self.groups:
        for cell, value in group.cells.items():
            if value is not None:

                #Se a mesma célula tiver duas pistas diferentes, o
problema é impossível.
                if cell in fixed and fixed[cell] != value: #nao ha
contradicao

                    return None

                fixed[cell] = value

    return fixed

def _initial_domains(self): #conceito csp
    #cria os domínios iniciais de todas as células

    fixed = self._fixed_values()

    if fixed is None:
        return None

    domains = {} #conj de val q a variavel pode assumir

    valores = set(range(1, self.N + 1)) # de 1 - n2

    for i in range(self.N):
        for j in range(self.N):
            cell = (i, j) #cria as celulas

            if cell in fixed:
                domains[cell] = {fixed[cell]} #se for pista o domain é
a pista
            else:
                domains[cell] = set(valores) #senao é 1-n2

    return domains

def _propagate(self, domains):
    #remove dos domínios os valores já usados por células fixadas, e
possib imp

```

```

        changed = True

        while changed: #while pq ao tirar valor de uma celula da nova info
            aos outros
            changed = False

            for cell in domains:

                #uma célula já com um único valor não precisa de ser
                alterada

                if len(domains[cell]) != 1:
                    continue #mais q um nr n sabemos valor, um nr sabemos

                value = next(iter(domains[cell]))

                for neighbor in self._neighbors(cell):

                    #só remove o valor de vizinhos que ainda tenham várias
                    possibilidades., nao podem ter a possibilidade da anterior
                    if len(domains[neighbor]) > 1:
                        if value in domains[neighbor]:
                            domains[neighbor].remove(value) #retira o n
                            deles, propagacao de restricoes
                            changed = True

                        #domínio vazio -> contradição, é impossivel
                        if len(domains[neighbor]) == 0:
                            return False

                #verificar se duas células do mesmo grupo ficaram com o mesmo valor
                fixo

                for group in self.groups:
                    usados = set()

                    for cell in group.cells:
                        if len(domains[cell]) == 1:
                            value = next(iter(domains[cell]))

                            if value in usados:
                                return False

                            usados.add(value)

                return True

    def _select_cell(self, domains):
        #Escolhe a célula livre com menos possibilidades MRV, escolher qual
        celula a preencher a seguir

        candidates = [
            cell
            for cell in domains
            if len(domains[cell]) > 1
        ]

        if not candidates:

```

```

        return None

    return min(
        candidates,
        key=lambda cell: len(domains[cell]) #MRV minimum remaining values, célula com menos poss restante
    )

    def _backtrack(self, domains):
        #Resolve recursivamente o CSP

        if not self._propagate(domains):
            return None

        cell = self._select_cell(domains) #escolhe com mrv

        #não existem mais células por preencher.
        if cell is None:
            return domains

        #experimentar cada possibilidade.
        for value in sorted(domains[cell]):

            new_domains = {
                c: set(values)
                for c, values in domains.items() #cria copia pq nao se quer destruir o original
            }

            new_domains[cell] = {value} #escolhe

            result = self._backtrack(new_domains) #assume n e tenta resolver o resto

            if result is not None:
                return result #se resulta resulta senao backtracking

        return None

    def solve(self):
        """Resolve o CSP.

        Devolve a grelha preenchida ou None se não houver solução.
        """

        domains = self._initial_domains() #cria

        if domains is None:
            return None

        result = self._backtrack(domains) #manda resolver

        if result is None:
            return None #nao ha resol

        return [

```

```

        [
            next(iter(result[(i, j)]))
            for j in range(self.N) #se houver transforma numa matriz
        ]
        for i in range(self.N)
    ]

```

Célula de teste R5

```

In [ ]: _modelo = SudokuCSP(3)

_grupo = Grupo(3)

_grupo.add(0, 0, 1)
_grupo.add(0, 1, 2)
_grupo.add(0, 2, 3)

_modelo.add_group(_grupo)

_solucao = _modelo.solve()

assert _solucao is not None

assert _solucao[0][0] == 1
assert _solucao[0][1] == 2
assert _solucao[0][2] == 3

# n=2
_modelo2 = SudokuCSP(2)

_grupo2 = Grupo(2)

_grupo2.add(0, 0, 1)
_grupo2.add(0, 1, 2)

_modelo2.add_group(_grupo2)

_solucao2 = _modelo2.solve()

assert _solucao2 is not None

assert _solucao2[0][0] == 1
assert _solucao2[0][1] == 2

mo.callout(
    mo.md("`SudokuCSP` OK para $n=3$ e $n=2$."),
    kind="success"
)

```

SudokuCSP OK para $n = 3$ e $n = 2$.

R5: modelo CSP e técnica de resolução

Correspondência com o enunciado: modelo e resolução → `SudokuCSP`.

Variáveis e domínios: Cada célula (i, j) é uma variável com domínio $\{1, \dots, n^2\}$. Uma célula fixa por um grupo (uma pista) tem o domínio reduzido a esse único valor. Se a mesma célula for fixada com dois valores diferentes por grupos distintos, o problema é impossível e `solve` devolve `None` de imediato.

Restrições: Cada grupo impõe que as suas células tenham valores todos diferentes. Dizemos que duas células são vizinhas se partilham pelo menos um grupo; assim, a restrição "todos diferentes" num grupo equivale a exigir valores diferentes em cada par de células vizinhas. Os vizinhos de cada célula são calculados uma só vez e guardados, porque são consultados diversas vezes.

Técnica escolhida: Resolvemos o CSP da seguinte maneira:

1. **Propagação de restrições:** quando uma célula fica com um único valor possível, esse valor é retirado dos domínios de todos os seus vizinhos, e repete-se enquanto houver alterações.
2. **Heurística MRV** (*minimum remaining values*): quando é preciso escolher, escolhemos a célula ainda por decidir com **menos** valores possíveis. Assim falha-se cedo, e as escolhas com mais probabilidade de dar conflito são feitas primeiro.
3. **Retrocesso** (*backtracking*): a propagação sozinha só preenche o que é *forçado*. Quando fica algum domínio com mais de um valor, não há mais deduções diretas, por isso experimentamos cada valor da célula escolhida (numa cópia dos domínios), propagamos, e voltamos atrás se se chegar a uma contradição.

Há uma contradição quando duas células vizinhas ficam decididas com o mesmo valor. Se nenhum valor da célula escolhida resultar, o ramo é abandonado, e se todos falharem desde o início `solve` devolve `None`, o que sinaliza de forma distinguível que não há solução.

A razão desta abordagem: O enunciado deixa a técnica ao nosso critério.

Implementámos o CSP diretamente, em vez de usar uma biblioteca (CP-SAT ou SCIP), porque assim temos controlo sobre cada passo e percebemos o que acontece. O que importa para o enunciado é que o modelo **só usa as células dos grupos** (`group.cells`): não pergunta se um grupo é um `Cube`, um `Path` ou uma pista, e portanto qualquer grupo novo entra sem mexer no modelo. O `add_group` aceita um número arbitrário de grupos e rejeita grupos com um n diferente do do modelo.

Limites de desempenho: A propagação só elimina valores a partir de células já decididas e o retrocesso copia todos os domínios em cada ramo, o que fica caro

quando há n^4 células. Um solver com restrições `AllDifferent` (como o CP-SAT) faz propagação muito mais forte e escalaria melhor.

R6- Montar e resolver o Sudoku

```
In [ ]: def sudoku(n, k=None):
    """cria e resolve um Sudoku  $n^2 \times n^2$  com pistas aleatórias."""

    N = n * n

    #criar modelo CSP
    modelo = SudokuCSP(n)

    #linhas
    for i in range(N):
        modelo.add_group(
            Path(n, (i, 0), (i, N - 1))
        )

    #colunas
    for j in range(N):
        modelo.add_group(
            Path(n, (0, j), (N - 1, j))
        )

    #blocos  $n \times n$ 
    for bi in range(n):
        for bj in range(n):
            modelo.add_group(
                Cube(n, bi, bj)
            )

    #pistas aleatórias
    grupo_pistas = pistas(n, k)

    modelo.add_group(grupo_pistas)

    #resolver
    solucao = modelo.solve()

    return solucao, grupo_pistas
```

Célula de Testes R6

```
In [ ]: solucao, grupo_pistas = sudoku(3)

assert solucao is not None

html = """
<table style="
    border-collapse: collapse;
    margin: auto;
    font-size: 20px;
    text-align: center;
```

```

">
"""

for i in range(9):
    html += "<tr>"

    for j in range(9):
        #linhas entre blocos 3x3
        border_right = "3px solid black" if j in (2, 5, 8) else "1px solid
black"
        border_bottom = "3px solid black" if i in (2, 5, 8) else "1px solid
black"

        html += f"""
        <td style="
            width: 35px;
            height: 35px;
            border-right: {border_right};
            border-bottom: {border_bottom};
            border-left: 1px solid black;
            border-top: 1px solid black;
        ">
            {solucao[i][j]}
        </td>
        """

    html += "</tr>"

html += "</table>"

mo.Html(html)

```

4	9	5	1	2	3	7	6	8
1	2	3	8	6	7	4	5	9
6	7	8	4	5	9	1	2	3
2	3	1	6	7	4	8	9	5
8	6	9	3	1	5	2	4	7
5	4	7	2	9	8	3	1	6
3	5	2	7	4	6	9	8	1
7	1	6	9	8	2	5	3	4
9	8	4	5	3	1	6	7	2

Justificação R6

Correspondência com o enunciado: Sudoku completo $\rightarrow$ `sudoku` .

O que faz. `sudoku(n, k)` monta e resolve um Sudoku $n^2 \times n^2$ com pistas aleatórias, juntando num único modelo `SudokuCSP` :

- as n^2 **linhas**, cada uma um `Path` de $(i, 0)$ a $(i, N - 1)$;
- as n^2 **colunas**, cada uma um `Path` de $(0, j)$ a $(N - 1, j)$;
- os n^2 **blocos** $n \times n$, cada um um `Cube` de índices (b_i, b_j) ;
- **um** grupo de pistas aleatórias, gerado por `pistas` .

São $3n^2 + 1$ grupos no total. Depois chama `solve` , que devolve a grelha preenchida ou `None` .

Porque é que o modelo não distingue a origem dos grupos: Todos os grupos entram com o mesmo método `add_group` , e o modelo só usa as células de cada um. Linhas, colunas, blocos e pistas são tratados da mesma forma, que é o que o enunciado pede em R5 e mostra a utilidade da abstração `Grupo` . A ordem em que os

grupos são acrescentados não altera o resultado. Para acrescentar uma variante bastaria juntar mais grupos aqui, sem mexer no modelo.

Parametrização: Todos os ciclos dependem só de n (e de $N = n^2$), por isso nada está fixo a 9×9 : a mesma função resolve a grelha 4×4 ($n = 2$), a 9×9 ($n = 3$) ou outra.

Valor devolvido: A função devolve o par `(solucao, grupo_pistas)` e não só a solução, porque a validação precisa de saber quais eram as pistas para confirmar que se mantêm na solução.

Puzzle sem solução: Se as pistas aleatórias derem um puzzle impossível, `solve` devolve `None` e `sudoku` limita-se a devolvê-lo, sem repetir o sorteio (justificada na R4). Quem chama a função tem de tratar esse caso, e é o que fazem os testes e a apresentação da grelha.

Validar automaticamente

```
In [ ]: def validar_sudoku(solucao, n, grupo_pistas):
    """valida uma solução de Sudoku"""

    N = n * n
    esperado = set(range(1, N + 1))

    #verificar dimensões
    assert len(solucao) == N
    assert all(len(linha) == N for linha in solucao)

    #verificar valores
    assert all(
        valor in esperado
        for linha in solucao
        for valor in linha
    )

    #verificar linhas
    for i in range(N):
        assert set(solucao[i]) == esperado

    #verificar colunas
    for j in range(N):
        coluna = {
            solucao[i][j]
            for i in range(N)
        }

        assert coluna == esperado

    #verificar blocos
    for bi in range(n):
        for bj in range(n):

            bloco = {
```

```

        solucao[bi * n + di][bj * n + dj]
        for di in range(n)
        for dj in range(n)
    }

    assert bloco == esperado

#verificar pistas
    for (i, j), valor in grupo_pistas.cells.items():
        if valor is not None:
            assert solucao[i][j] == valor

    return True

```

Validação automática

`validar_sudoku` verifica, para uma grelha resolvida, tudo o que o enunciado pede: que as dimensões são $n^2 \times n^2$ e que os valores estão em $[1, n^2]$; que cada linha, cada coluna e cada bloco $n \times n$ contêm exatamente os valores $1, \dots, n^2$, sem repetições; e que as células fixadas pelas pistas mantêm o valor com que foram fixadas. O teste final corre o fluxo completo (gerar pistas, montar linhas, colunas, blocos e pistas, resolver, validar) com $n = 2$ e $n = 3$, para mostrar que nada está fixo a 9×9 .